

Grid-Guard Mini For House

I went through the `GridGuard.ipynb` notebook itself, including all 8 code cells and their recorded outputs, and I also used the accompanying EnergyGuard/GridGuard build guide to connect the implementation to the intended theory and architecture. The guide explicitly frames the project around supervised forecasting, unsupervised anomaly detection, Non-IID data, Federated Learning, SHAP, an agent layer, and then API/dashboard integration.

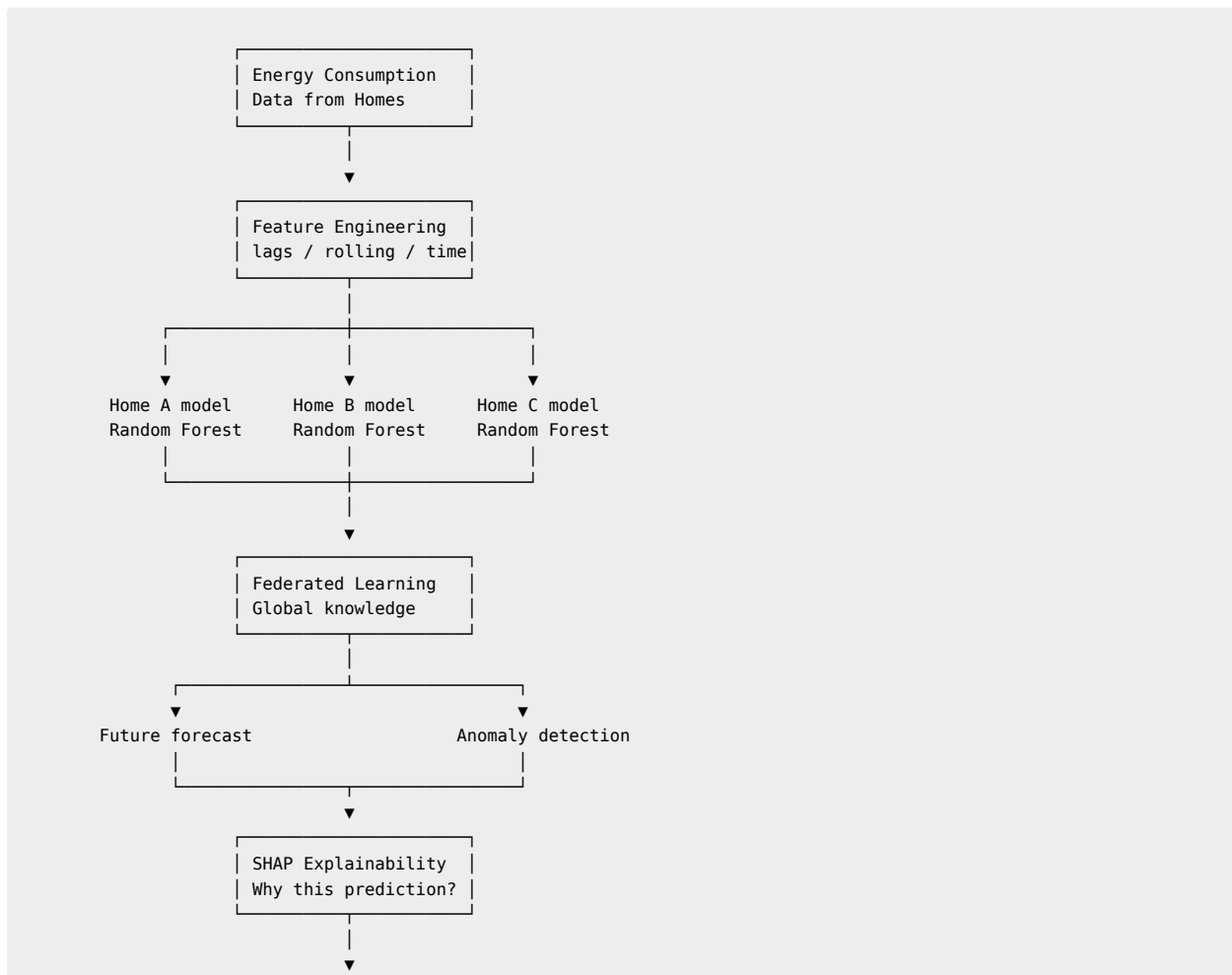
There is one very important distinction to keep in mind throughout this explanation:

The notebook is an educational prototype. Several parts demonstrate the idea of Federated Learning and forecasting, but they are not production-grade implementations of those concepts.

In fact, there are a few important technical issues in the notebook that are worth understanding because they will matter in an FYP defense.

1. What this whole project is trying to do

The complete conceptual pipeline is:



Energy Agent
Observe → Reason →
Act

The accompanying guide describes the same overall progression: data → Random Forest forecasting → Isolation Forest → Non-IID homes → Federated Learning → SHAP → agent → API → dashboard.

Your actual notebook contains the implementation through the **Energy Agent**. The separate guide continues the architecture with FastAPI and Streamlit, but those are not separate executable cells in the notebook you uploaded.

2. The three most important ideas in the entire project

Before going line-by-line, understand these three ideas.

2.1 Forecasting

Forecasting means:

Given what I know about the past and current context, what will energy consumption be?

Mathematically, you can think of it as:

$$\hat{y}_{t+1} = f(X_t)$$

where:

- X_t = features available at time t
- y_{t+1} = future consumption
- $\hat{y}_{t+1}$ = predicted future consumption
- f = machine-learning model

Your project uses a **Random Forest Regressor**.

2.2 Anomaly detection

Anomaly detection asks a different question:

Does this consumption pattern look unusual compared with what is normally observed?

It is not primarily trying to predict the next value.

It tries to find:

$$x \neq \text{normal distribution}$$

Your project uses **Isolation Forest**, which is an unsupervised anomaly-detection algorithm.

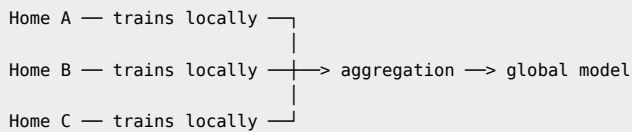
The guide explicitly distinguishes forecasting as supervised learning and anomaly detection as unsupervised learning.

2.3 Federated Learning

Federated Learning asks:

Can multiple homes collaboratively improve a machine-learning model without sending their raw training data to a central server?

Conceptually:



The fundamental idea is that the **data stays on the client**, while some form of model update is communicated.

That is the privacy-preserving motivation described in the guide.

3. Cell 0 — Installing libraries

Your first notebook cell is essentially:

```
!pip install numpy pandas scikit-learn matplotlib
```

This installs four major libraries.

NumPy

```
import numpy as np
```

NumPy provides:

- arrays
- mathematical functions
- random number generation
- vectorized numerical computation

Examples in your notebook:

```
np.random.seed(42)
np.sin(...)
np.pi
np.random.normal(...)
np.random.choice(...)
np.random.uniform(...)
```

Pandas

```
import pandas as pd
```

Pandas is used for tabular data.

The central object is the:

DataFrame

You can think of a DataFrame as an Excel table inside Python.

For example:

timestamp	consumption_kwh	hour
2024-01-01 00:00	0.8	0
2024-01-01 00:15	0.7	0
2024-01-01 00:30	0.9	0

Scikit-learn

This contains the machine-learning models:

RandomForestRegressor
IsolationForest

and metrics such as:

mean_absolute_error
mean_squared_error

Matplotlib

Used for visualizing:

- energy consumption
- predictions
- anomalies
- convergence
- SHAP-like plots

Important notebook issue

Your first cell does **not install** `shap`, even though later cells contain:

```
import shap
```

So on a completely fresh environment, the notebook could fail at the SHAP cell unless SHAP is already installed.

The project guide, however, lists SHAP among the required packages.

4. Cell 1 — Synthetic smart-home data generation

This cell creates the entire data universe for the experiment.

There is no real smart-meter dataset here.

Instead, the notebook creates **synthetic energy consumption data**.

That is actually a reasonable educational choice because it lets you control:

- daily patterns
- home differences
- noise
- anomalies
- sample size

The guide also explicitly says that the project uses realistic synthetic data.

5. Creating timestamps

The notebook uses:

```
timestamps = pd.date_range(  
    start='2024-01-01',  
    end='2024-03-01',  
    freq='15min'  
)
```

This means:

- start: January 1, 2024
- end: March 1, 2024
- interval: 15 minutes

There are four observations per hour:

$$60 \div 15 = 4$$

and therefore:

$$4 \times 24 = 96$$

observations per day.

For 60 days:

$$96 \times 60 = 5760$$

But the notebook output says:

```
Total timestamps: 5761
```

Why?

Because `pd.date_range()` includes the endpoint by default.

So the sequence contains:

```
Jan 1 00:00  
...  
Feb 29 23:45  
Mar 1 00:00
```

That final endpoint adds one extra timestamp.

This is a small but useful Python/Pandas detail.

6. `np.random.seed(42)`

You have:

```
np.random.seed(42)
```

This is extremely important for reproducibility.

Normally:

```
np.random.normal(...)
```

produces different random values each time.

By fixing the seed:

```
42
```

you get the same pseudo-random sequence each run.

This means:

```
Run 1 → same synthetic dataset  
Run 2 → same synthetic dataset  
Run 3 → same synthetic dataset
```

That is useful for debugging and FYP demonstrations.

7. `generate_home_data()`

This is the main data generator.

Conceptually:

```
def generate_home_data(  
    timestamps,  
    base_load,  
    noise_level,  
    name  
):
```

Each home is generated from three things:

Base load

How much energy the home generally consumes.

Noise level

How unpredictable/variable the household is.

Time

Consumption changes throughout the day.

8. `hour = ts.hour`

For each timestamp:

```
hour = ts.hour
```

gives:

```
00, 01, 02, ..., 23
```

This becomes an important machine-learning feature later.

9. `day = ts.dayofweek`

This gives:

```
0 = Monday  
1 = Tuesday  
2 = Wednesday  
3 = Thursday  
4 = Friday  
5 = Saturday  
6 = Sunday
```

That becomes another feature.

10. The synthetic daily energy pattern

The notebook creates:

```
hourly_pattern = (  
    0.5 * np.sin(2 * np.pi * (hour - 6) / 24)  
    + 0.3 * np.sin(2 * np.pi * (hour - 19) / 12)  
    + 1.0  
)
```

This is one of the most mathematically interesting pieces of the notebook.

You are using **sinusoidal functions** to simulate periodic behavior.

Why?

Because electricity consumption has temporal patterns:

night	→ lower
morning	→ higher
afternoon	→ moderate
evening	→ higher
night	→ lower

The sine wave provides a smooth periodic structure.

First sine component

```
0.5 * np.sin(2 * np.pi * (hour - 6) / 24)
```

The denominator `24` creates roughly a 24-hour cycle.

Second sine component

```
0.3 * np.sin(2 * np.pi * (hour - 19) / 12)
```

Adds another periodic component with a shorter cycle.

```
+1.0
```

Moves the overall curve upward so the value is centered around a positive baseline.

Important subtlety

The comments say that the pattern peaks around morning/evening.

But the exact formula does not mathematically produce only a clean 8 AM and 7 PM peak.

The actual hourly curve is the result of **two sine waves being added together**.

That is called a superposition of periodic components.

This is a synthetic approximation, not a physical electricity-consumption model.

11. Weekend factor

The notebook has:

```
weekend_factor = 1.1 if day >= 5 else 1.0
```

Therefore:

```
Weekday → ×1.0  
Weekend → ×1.1
```

The intended interpretation is:

People may spend more time at home on weekends.

Thus:

$\text{Consumption} = \text{BaseLoad} \times \text{TimePattern} \times \text{WeekendFactor} + \text{NoiseConsumption} = \text{BaseLoad} \times \text{TimePattern} \times \text{WeekendFactor} + \text{Noise}$

12. Random noise

Then:

```
np.random.normal(0, noise_level)
```

adds Gaussian noise.

This is:

$\epsilon \sim N(0, \sigma)$ \epsilon \sim N(0, \sigma)

where:

- mean = 0
- standard deviation = `noise_level`

For example:

```
noise_level=0.15
```

means relatively stable consumption.

While:

```
noise_level=0.5
```

means much more variability.

13. The three homes

This is where the project creates **Non-IID data**.

Home A

```
base_load=1.5  
noise_level=0.15
```

Moderate consumption and low variability.

Home B

```
base_load=3.5  
noise_level=0.35
```

Higher average load.

Home C

```
base_load=2.0  
noise_level=0.5
```

Moderate load but greater variability.

The actual recorded averages were approximately:

```
Home A → 1.54 kWh  
Home B → 3.59 kWh  
Home C → 2.07 kWh
```

This is the notebook's artificial Non-IID setup.

14. What exactly does Non-IID mean?

IID means:

| Independent and Identically Distributed.

In a simplified textbook scenario, all clients might have data generated from approximately the same distribution:

$$P_A(X,Y) \approx P_B(X,Y) \approx P_C(X,Y) \approx P_A(X,Y) \approx P_B(X,Y) \approx P_C(X,Y)$$

Non-IID means:

$$P_A(X,Y) \neq P_B(X,Y) \neq P_C(X,Y) \quad P_A(X,Y) \neq P_B(X,Y) \neq P_C(X,Y)$$

For your notebook:

```
Home A → lower load  
Home B → higher load  
Home C → more variable load
```

So the clients do not all look statistically identical.

The guide specifically identifies Non-IID data as a major challenge of Federated Learning.

15. Why Non-IID matters so much in Federated Learning

Imagine training on:

```
Home A → mostly 1–2 kWh  
Home B → mostly 3–5 kWh  
Home C → 1–3 kWh but highly noisy
```

Now suppose the server wants one global model.

There is a tension:

```
Model optimized for A  
    ↓  
may be bad for B  
  
Model optimized for B  
    ↓  
may be bad for A
```

This is called **client heterogeneity** or **statistical heterogeneity**.

It is one of the central challenges in real FL.

16. Injecting anomalies

The notebook creates artificial anomalies using:

```
df['is_anomaly'] = 0
```

and then randomly selects rows:

```
anomaly_indices = np.random.choice(...)
```

For every selected row:

```
df.at[idx, 'consumption_kwh'] = np.random.uniform(8, 15)
df.at[idx, 'is_anomaly'] = 1
```

So:

```
Normal:
~0.5 - 4 kWh

Anomaly:
8 - 15 kWh
```

This is intentionally obvious.

The anomalies are designed to simulate events such as:

- faulty appliance
- unusually heavy load
- device left on
- unusual consumption behavior

The label:

```
is_anomaly
```

is the **ground truth**.

That distinction is crucial.

17. Ground truth vs prediction

You later have:

```
y_true
```

and:

```
y_pred
```

`y_true` is what actually happened.

`y_pred` is what the anomaly model thinks happened.

That allows you to calculate:

- precision
- recall
- F1

- confusion matrix

Without ground-truth labels, you cannot compute standard supervised classification metrics in the same way.

18. Feature engineering

This is one of the most important parts of the project.

Raw consumption:

```
0.8
0.7
0.9
1.2
...
```

does not provide enough contextual information.

So the notebook creates:

```
lag_1
lag_4
lag_96
rolling_mean_4
rolling_mean_96
rolling_std_4
hour
day_of_week
is_weekend
month
```

The guide describes lag and rolling features as methods for giving the model historical context.

19. What is a lag feature?

Suppose consumption is:

```
t-3  1.0
t-2  1.2
t-1  1.4
t    1.6
```

Then:

```
lag_1
```

means:

$\text{lag}_1(t) = y_{t-1}$

So:

$$\text{lag}_1 = 1.4$$

20. **lag_4**

Since measurements happen every 15 minutes:

$$1 \text{ step} = 15 \text{ minutes}$$

Therefore:

$$4 \text{ steps} = 60 \text{ minutes}$$

So:

$$\text{lag}_4$$

means:

| consumption one hour ago.

Mathematically:

$$\text{lag}_4(t) = y_t - 4 \text{lag}_4(t) = y_{t-4}$$

21. **lag_96**

Because:

$$96 \times 15 \text{ minutes} = 1440 \text{ minutes} = 24 \text{ hours}$$

therefore:

$$\text{lag}_{96}$$

means:

| consumption exactly 24 hours ago.

This is extremely useful in energy forecasting because yesterday's consumption at the same time often contains useful information.

22. Rolling mean

The notebook uses:

```
df['rolling_mean_4'] =  
    df['consumption_kwh'].rolling(4).mean()
```

A rolling mean is a moving average.

Conceptually:

RollingMean $=y_t+y_{t-1}+y_{t-2}+y_{t-3}$ RollingMean $_t = \frac{y_t+y_{t-1}+y_{t-2}+y_{t-3}}{4}$

It smooths short-term fluctuations.

Similarly:

```
rolling_mean_96
```

represents the recent 96-point average.

Since 96 points = 24 hours, it represents roughly a one-day window.

23. Rolling standard deviation

```
rolling_std_4
```

measures short-term variability.

If recent values are:

```
1.0  
1.1  
1.0  
1.1
```

the standard deviation is small.

If they are:

```
0.5  
3.0  
0.8  
4.0
```

the standard deviation is much larger.

So `rolling_std_4` gives the model an idea of:

| "How unstable has consumption been recently?"

24. `dropna()`

Lagging creates missing values.

For example, at the very beginning:

```
lag_1 → NaN  
lag_4 → NaN  
lag_96 → NaN
```

because there is no historical data before the first observation.

Therefore:

```
df = df.dropna()
```

removes incomplete rows.

That is why:

```
5761 original rows
```

becomes:

```
5665 rows
```

The difference is:

$5761 - 5665 = 96$

because the 96-step lag requires the first 96 positions to have sufficient history.

25. Important forecasting problem in the notebook

This is one of the biggest technical issues you should understand.

The notebook says it is predicting the **next energy consumption value**.

But the target is:

```
TARGET_COLUMN = 'consumption_kwh'
```

There is no:

```
.shift(-1)
```

to create:

y_{t+1}

So technically, the model is trained to predict:

y_t

not explicitly:

$y_t + 1y_{t+1}$

That means the comments and actual implementation do not fully match.

26. Even more important: target leakage

There is a second and more serious issue.

The rolling features are created directly from:

```
consumption_kwh
```

and include the current row.

For example:

```
rolling_mean_4
```

contains:

$y_t + y_{t-1} + y_{t-2} + y_{t-3} \frac{1}{4}$

But the target is also:

y_t

Therefore the model has indirect access to part of its own answer.

That is **target leakage**.

The same issue affects:

```
rolling_mean_4  
rolling_mean_96  
rolling_std_4
```

because they are calculated using the current observation.

27. Why this matters

Suppose:

```
Actual target = 5.0 kWh
```

and:

```
rolling_mean_4 = 4.5
```

The model is being told:

"The target is probably around 4.5"

But that 4.5 was calculated using the target itself.

So the model's job has become artificially easier.

That can explain why:

```
rolling_mean_4
```

becomes extremely dominant in the model importance and SHAP results.

28. How proper forecasting should work

For one-step-ahead forecasting, a better setup is:

```
Features at time t
      ↓
Predict consumption at time t+1
```

For example:

```
df['target_next'] = df['consumption_kwh'].shift(-1)
```

Then feature values must represent information available at time t , not information from $t+1$.

Conceptually:

$X_t = [y_t, y_{t-1}, y_{t-24h}, \text{rolling_t}, \text{hour}_t, \dots]$

and:

$y_{t+1} = \text{target}_t$

This is much closer to genuine forecasting.

29. Cell 2 — Random Forest forecasting

Now we reach the supervised-learning component.

The notebook uses:

```
RandomForestRegressor
```

A Random Forest is an ensemble of Decision Trees.

The guide describes a forest as many decision trees whose predictions are combined for the final regression output.

30. What is a Decision Tree?

A decision tree repeatedly asks questions.

For example:

```
Is rolling_mean_4 > 2.5?  
  /   \  
Yes    No  
  /   \  
Is hour > 18?  Is weekend?
```

Eventually it reaches a leaf containing a prediction.

For regression, the leaf generally represents an average target value among training samples that reached that leaf.

31. Why use many trees?

One tree may be unstable.

Suppose:

```
Tree 1 → 2.8  
Tree 2 → 3.1  
Tree 3 → 2.9  
Tree 4 → 3.0  
Tree 5 → 2.7
```

The forest averages them:

$$\hat{y} = \frac{2.8 + 3.1 + 2.9 + 3.0 + 2.7}{5} = 2.9$$

So the final prediction is an ensemble result.

32. Why Random Forest works well with this data

Random Forest is useful here because:

- it handles nonlinear relationships
- it does not require feature scaling
- it can model interactions
- it handles mixed numerical features
- it can capture threshold behavior
- it is relatively easy to interpret compared with deep neural networks

For example:

```
hour
```

could interact with:

```
rolling_mean_4
```

and:

```
is_weekend
```

in a nonlinear way.

33. The feature list

Your model gets 10 features:

```
FEATURE_COLUMNS = [  
    'lag_1',  
    'lag_4',  
    'lag_96',  
    'rolling_mean_4',  
    'rolling_mean_96',  
    'rolling_std_4',  
    'hour',  
    'day_of_week',  
    'is_weekend',  
    'month'  
]
```

Conceptually:

Feature	Meaning
lag_1	previous 15-minute consumption
lag_4	consumption 1 hour ago
lag_96	consumption 24 hours ago
rolling_mean_4	recent short-term average
rolling_mean_96	recent daily average
rolling_std_4	recent variability
hour	time of day
day_of_week	weekday position
is_weekend	weekday/weekend
month	month number

34. **X** and **y**

You have:

```
X = df[FEATURE_COLUMNS].values
```

This is the input matrix.

If you have:

```
5665 samples  
10 features
```

then:

$X \in \mathbb{R}^{5665 \times 10}$

The notebook confirms:

```
Feature matrix shape: (5665, 10)
```

Then:

```
y = df[TARGET_COLUMN].values
```

is:

$y \in \mathbb{R}^{5665}$

35. Time-based train/test split

This part is conceptually correct.

The notebook does:

```
split_idx = int(len(df) * 0.8)  
  
X_train = X[:split_idx]  
X_test = X[split_idx:]  
  
y_train = y[:split_idx]  
y_test = y[split_idx:]
```

So:

```
first 80% → training  
last 20% → testing
```

The guide specifically recommends this time-based separation because random splitting would allow future observations to leak into training.

36. Why random train/test splitting is dangerous for time series

Suppose:

```
January → training  
February → testing
```

That represents reality:

```
past → train  
future → test
```

But if you randomly shuffle:

```
Jan 3 → train  
Feb 15 → train  
Jan 20 → test  
Feb 1 → train  
...
```

then the model is effectively learning from future conditions.

That violates the forecasting scenario.

This is called **temporal leakage** or **look-ahead bias**.

37. Random Forest hyperparameters

The notebook uses:

```
RandomForestRegressor(  
    n_estimators=100,  
    max_depth=10,  
    random_state=42,  
    n_jobs=-1  
)
```

n_estimators=100

Build 100 trees.

More trees usually reduce ensemble variance, at the cost of more computation.

max_depth=10

Each tree can grow to a maximum depth of 10.

This limits complexity.

Without limits, trees can become very complex and memorize training behavior.

random_state=42

Makes the result reproducible.

n_jobs=-1

Uses all available CPU cores.

38. **model.fit()**

This is the actual training step:

```
model.fit(X_train, y_train)
```

This is where the Random Forest learns relationships between:

```
features → consumption
```

Conceptually:

$f(X) \rightarrow y_f(X) \rightarrow y$

39. Predictions on training and test sets

You have:

```
y_pred_train = model.predict(X_train)
y_pred_test = model.predict(X_test)
```

Why both?

Because the comparison helps identify overfitting.

40. MAE

MAE means:

| Mean Absolute Error

Formula:

$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$ | $MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$

Suppose actual values are:

3
5
4

and predictions are:

2
4
5

Errors:

1
1
1

MAE:

$$1+1+1=3 \Rightarrow \frac{1+1+1}{3}=1$$

So MAE is easy to explain:

| On average, how many kWh away are my predictions?

The guide explains the metric the same way.

41. RMSE

RMSE means:

| Root Mean Squared Error

Formula:

$$RMSE = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$$

The important difference:

MAE

Treats errors linearly.

RMSE

Squares the errors first.

Therefore a very large error is punished more heavily.

For example:

```
Errors:
0.5
0.5
5.0
```

MAE does not distinguish the 5 nearly as dramatically as RMSE does.

42. Why use both MAE and RMSE?

They tell slightly different stories.

MAE

Good for:

- | Typical prediction error.

RMSE

Good for:

- | How badly do large mistakes hurt us?

In an energy-management system, large prediction errors can matter significantly because they may affect load decisions.

43. Overfitting

Overfitting occurs when:

```
Model memorizes training data
```

instead of learning general patterns.

A classic sign:

```
Train error → very low
Test error  → much higher
```

The guide explains this principle directly.

44. Your overfitting rule

The notebook checks:

```
if train_mae < test_mae * 0.5:
```

That means:

| If training MAE is less than half the test MAE, issue a warning.

This is a **heuristic**, not a universal statistical rule.

It is okay for an educational demonstration, but you should not claim:

| "This proves there is no overfitting."

It does not.

45. Notebook forecasting results

The notebook recorded approximately:

```
Home A
Train MAE = 0.0994
Test MAE  = 0.1643
Test RMSE = 0.5600

Home B
Train MAE = 0.1910
Test MAE  = 0.2808
Test RMSE = 0.5344

Home C
Train MAE = 0.2594
Test MAE  = 0.3942
Test RMSE = 0.6734
```

The important conceptual point is that the test metrics measure generalization to the held-out time period.

However, because of the feature-leakage problem discussed earlier, these numbers should **not** be treated as rigorous one-step-ahead forecasting performance.

46. Feature importance

Random Forest provides:

```
model.feature_importances_
```

This produces values indicating how much each feature contributed to the tree-splitting process used by the forest.

In the notebook, `rolling_mean_4` dominates.

For example, Home A reports approximately:

```
rolling_mean_4 → 0.602
hour           → 0.103
lag_96         → 0.075
```

```
rolling_std_4    → 0.067
lag_1            → 0.064
```

Again, the dominance of `rolling_mean_4` should be interpreted carefully because that feature contains the current target.

47. Saving the model

The notebook uses:

```
pickle.dump(model, f)
```

and saves:

```
models/model_a.pkl
models/model_b.pkl
models/model_c.pkl
```

A `.pkl` file stores the Python object so it can later be loaded without retraining.

Then:

```
pickle.load(f)
```

reconstructs the model.

48. Why model persistence matters

Without persistence:

```
Every API request
      ↓
train model again
```

which would be extremely inefficient.

Instead:

```
Training
      ↓
model.pkl
      ↓
load model
      ↓
predict
```

This is the basic idea behind deploying ML models.

49. Cell 3 — Isolation Forest anomaly detection

Now the project changes ML tasks.

Forecasting:

```
supervised
```

Anomaly detection:

```
unsupervised
```

The guide explicitly describes anomaly detection as learning what "normal" looks like without requiring labels for training.

50. How Isolation Forest works

Isolation Forest uses random partitions.

Imagine points in a feature space:

Normal:

• • • • • • • •

Anomaly:

x

A random split might isolate the  immediately.

The dense normal points need multiple splits.

Therefore:

```
easy to isolate → suspicious  
hard to isolate → normal
```

That is the main idea behind Isolation Forest.

51. The anomaly features

The notebook uses:

```
anomaly_features = [  
    'consumption_kwh',  
    'rolling_mean_4',  
    'rolling_std_4',
```

```
'hour'  
]
```

So anomaly detection looks at:

1. actual consumption
2. recent average
3. recent variability
4. time of day

Notice that the anomaly model does not use the same feature set as the forecasting model.

That is intentional conceptually.

Forecasting needs historical prediction features.

Anomaly detection needs features describing unusual behavior.

52. `contamination=0.02`

The model uses:

```
contamination=0.02
```

meaning approximately:

2%2\%

of the data is expected to be anomalous.

The guide explains `contamination` as the expected fraction of anomalies.

This parameter has a huge effect.

Your dataset has 5665 rows.

$5665 \times 0.02 = 113.3$

and the model detected:

```
114 anomalies
```

That is exactly what we should expect from the contamination setting.

53. Why did it find 114 anomalies when only 20 were injected?

This is extremely important.

You created:

```
20 actual anomalies
```

but told the model:

```
expect roughly 2% anomalies
```

which is around:

```
113 observations
```

Therefore the model flags roughly 113 points.

The result:

```
Actual anomalies: 20  
Detected anomalies: 114
```

is therefore not surprising.

54. Confusion matrix

For Home A:

```
True Negative = 5551  
False Positive = 94  
False Negative = 0  
True Positive = 20
```

Let's understand every one.

True Positive

Actual anomaly and model says anomaly.

```
20
```

Excellent.

False Negative

Actual anomaly but model says normal.

```
0
```

So none of the injected anomalies were missed.

False Positive

Actually normal but model says anomaly.

94

This is where the model is over-alerting.

True Negative

Normal and correctly identified as normal.

5551

55. Precision

Precision asks:

| When the model raises an anomaly alert, how often is it actually an anomaly?

Formula:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Here:

$$\text{Precision} = \frac{20}{20 + 94} \approx 0.175$$

So about:

17.5%

That matches the notebook's approximately:

0.18

This means many alerts are false alarms.

56. Recall

Recall asks:

| Of all the actual anomalies, how many did we detect?

Formula:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Here:

$$20 \div 20 = 1.0 \frac{20}{20+0} = 1.0$$

So:

```
Recall = 100%
```

The system found every injected anomaly.

57. F1 score

F1 combines precision and recall:

$$F1 = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The result is around:

```
0.30
```

This makes sense because recall is excellent but precision is poor.

58. Very important interpretation

The anomaly model is therefore doing:

```
excellent recall  
poor precision
```

In an energy-monitoring system this means:

It is highly sensitive, but it generates many false alarms.

That may or may not be acceptable depending on the application.

59. Anomaly score

The notebook obtains:

```
model.score_samples(X)
```

Then performs a custom normalization:

```
1 - (  
    (score - score.min())  
    /
```

```
(score.max() - score.min())  
)
```

The result is scaled approximately:

```
0 → less anomalous  
1 → more anomalous
```

But very important:

| This normalized number is a custom score, not a true probability.

For example:

```
0.85
```

does **not** mean:

| 85% probability this is an anomaly.

It means that according to this normalization, the point is relatively far toward the anomalous end of the observed score range.

60. Another anomaly-model limitation

The anomaly model is fitted and evaluated on the same dataset:

```
model.fit(X)  
model.predict(X)
```

For an educational demo this is understandable.

For a rigorous evaluation, you would normally want something closer to:

```
historical/training normal data  
↓  
fit anomaly detector  
↓  
future held-out data  
↓  
evaluate
```

Otherwise the evaluation is less representative of deployment behavior.

61. Cell 4 — Local vs centralized learning

This cell demonstrates three concepts:

Local
Centralized
Federated

Federated is left for the next cell.

62. Local training

Each home trains independently:

```
model_a = train_rf(Xa_tr, ya_tr)
model_b = train_rf(Xb_tr, yb_tr)
model_c = train_rf(Xc_tr, yc_tr)
```

Conceptually:

```
Home A → Model A
Home B → Model B
Home C → Model C
```

Each model only learns from its home's data.

63. Centralized training

Then the notebook does:

```
X_all_tr = np.vstack([
    Xa_tr,
    Xb_tr,
    Xc_tr
])
```

and:

```
y_all_tr = np.concatenate([
    ya_tr,
    yb_tr,
    yc_tr
])
```

Now all three datasets are pooled together.

Conceptually:

```
Home A data ┌  
Home B data ┤ → central training dataset  
Home C data └
```

Then:

```
central_model = train_rf(X_all_tr, y_all_tr)
```

64. Why centralized learning can work well

The central model sees more examples.

Instead of:

```
4532 examples
```

for one home, it sees roughly:

$4532 \times 3 = 13596$ training examples.

So the global model can learn patterns shared across households.

But that comes with the privacy tradeoff:

| raw data has to be pooled centrally.

That is precisely the problem Federated Learning is intended to address.

65. Local vs centralized results

The notebook recorded:

Home A local	0.1643 MAE
Home A central	0.1654
Home B local	0.2808
Home B central	0.2890
Home C local	0.3942
Home C central	0.3900

The important lesson is not simply "central is better" or "local is better."

The real lesson is:

| With heterogeneous client distributions, a single global model may help some clients and not others.

That is exactly why Non-IID learning is interesting.

66. Cell 5 — Federated Learning

Now we reach the most important concept in your project.

Federated Learning architecture

In traditional centralized ML:

```
Client data
  ↓
Central server
  ↓
Train model
```

In FL:

```
Client data stays local

Home A — local training —┐
Home B — local training —┤→ server aggregation
Home C — local training —┘
```

The server should ideally never receive raw client training data.

The guide describes exactly this architecture.

67. FedAvg — the standard idea

FedAvg stands for:

| Federated Averaging.

Suppose client k produces local model parameters:

$w_{k,t}$

The server combines them.

The general FedAvg equation is:

$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{N} w_{k,t+1} = \sum_{k=1}^K \frac{n_k}{N} w_{k,t+1}$$

where:

- K = number of clients
- n_k = number of training examples on client k
- $N = \sum_k n_k$
- $w_{k,t+1}$ = client's updated parameters

If all clients have identical dataset sizes:

$$\frac{n_k}{N} = \frac{1}{K}$$

so it becomes an ordinary average.

The guide presents this basic averaging idea.

68. A critical issue: your notebook is not actually performing classical FedAvg

This is one of the most important things you should know for an FYP defense.

The notebook itself admits this:

```
# For traditional ML (not deep learning), we can't
# average the weights directly like in neural networks.
```

This is because a Random Forest does not have a simple vector of weights like a neural network.

69. Why Random Forest weights can't simply be averaged

For a neural network you can have:

```
Layer 1:
W1 = matrix

Layer 2:
W2 = matrix
```

Client A:

WAW_A

Client B:

WBW_B

Then conceptually:

$$W_{\text{global}} = \frac{W_A + W_B}{2}$$

That works because corresponding tensors have the same structure.

But Random Forest contains:

```
Tree 1:
  split feature = hour
  threshold = 18.3

Tree 2:
  split feature = lag_1
  threshold = 2.1
```

```
Tree 3:  
...
```

Client A's tree 1 is not necessarily equivalent to client B's tree 1.

The tree topology itself can differ.

So:

```
Tree A + Tree B
```

is not a simple element-wise numerical operation.

70. What your notebook actually does

The notebook uses a workaround.

Each client trains a Random Forest locally.

Then each client predicts on:

```
shared_X
```

The server gets:

```
client predictions
```

and computes:

```
averaged_predictions =  
    all_predictions.mean(axis=0)
```

So mathematically:

$$\bar{f}(x) = \frac{1}{K} \sum_{k=1}^K f_k(x)$$

This is **averaging model outputs**, not model weights.

71. Then the server trains another Random Forest

The server does:

```
self.global_model.fit(  
    shared_X,  
    averaged_predictions  
)
```

So:

```
Features
  ↓
Averaged local predictions
  ↓
Train global Random Forest
```

That is much closer to:

| ensemble prediction + model distillation

than classical FedAvg.

The server is essentially learning a model that approximates the average predictions of the local models.

72. What is knowledge distillation here?

Very roughly:

```
Local models
  ↓
teacher predictions
  ↓
average predictions
  ↓
global student model
```

The local models act like teachers.

The new global model learns to reproduce their averaged output.

This is a legitimate ML idea, but it should not be described as standard FedAvg weight averaging.

73. The biggest privacy problem in this implementation

The notebook does:

```
Xa_sample = Xa_tr[:sample_size]
Xb_sample = Xb_tr[:sample_size]
Xc_sample = Xc_tr[:sample_size]
```

then:

```
X_shared = np.vstack([
    Xa_sample,
    Xb_sample,
    Xc_sample
])
```

This means the server-side aggregation code receives actual feature rows from the client training datasets.

Those are:

```
raw private features
```

not merely model weights.

So although the architecture is intended to demonstrate:

| "raw data never leaves the home"

the actual notebook does not satisfy that requirement.

This is a very important distinction between the **concept** and the **implementation**.

The guide claims raw data remains local as the intended FL property, but this specific notebook's shared-sample implementation contradicts that claim.

74. `shared_y` is also interesting

The notebook creates:

```
y_shared = np.concatenate([
    ya_sample,
    yb_sample,
    yc_sample
])
```

but the aggregation function does not actually use `shared_y`.

So this variable is essentially unused.

That means:

```
shared_X → used
shared_y → not used
```

75. Another important problem: the initial global model doesn't exist

The notebook says:

```
Step 1: Initializing global model...
```

but it doesn't actually train a model during initialization.

It sets:

```
global_params = None
```

Then Round 1 calls:

```
client.local_train(global_params=None)
```

So Round 1 clients train entirely independently.

There is no initial global model being broadcast in that round.

Therefore the printed message:

```
Broadcasting global model...
```

is conceptually ahead of what the code is actually doing.

76. What happens in Round 1?

Because:

```
global_params=None
```

each client executes:

```
y_mixed = self.y_train
```

So:

```
Home A → train locally  
Home B → train locally  
Home C → train locally
```

Then the server aggregates their predictions.

Only after aggregation do we get:

```
global_params = {  
    'model': global_model  
}
```

So the actual sequence is:

```
Round 1:  
local models from scratch  
↓  
aggregate
```

```
↓
first global model

Round 2 onward:
global model exists
↓
clients use it
```

77. What happens after Round 1?

This part is unusual:

```
global_preds =
    global_params['model'].predict(self.X_train)
```

The client asks the global model:

| "What do you think my local training examples should be?"

Then it mixes:

```
y_mixed =
    0.7 * self.y_train +
    0.3 * global_preds
```

So:

$y_{mixed} = 0.7y_{local} + 0.3\hat{y}_{global}$

Then the client trains a new Random Forest on:

```
X_train
y_mixed
```

78. What does the 70/30 mixture mean?

It means:

```
70% influence → local ground truth
30% influence → global model prediction
```

This is intended to simulate the global model influencing local training.

But again:

| **This is not standard FedAvg.**

It is a custom pseudo-label/target-mixing strategy.

79. Why this can be problematic

Suppose the local truth says:

```
actual = 4.0
```

but the global model says:

```
global prediction = 2.5
```

Then:

$$y_{\text{mixed}} = 0.7(4.0) + 0.3(2.5) = 0.7(4.0) + 0.3(2.5) \\ = 2.8 + 0.75 = 3.55 = 2.8 + 0.75 = 3.55$$

So the local model is no longer learning the real target of 4.0.

It is learning:

```
3.55
```

This intentionally pulls the client toward the global model.

But with strong Non-IID distributions, that can hurt local performance.

80. The notebook results demonstrate this

Your federated local errors change over rounds.

For example:

Home B:

```
Round 1 → 0.2876  
Round 2 → 0.3731  
Round 3 → 0.3650  
Round 4 → 0.3771  
Round 5 → 0.3815
```

The local error becomes worse after global influence is introduced.

That is actually a very useful teaching result:

| A global model is not automatically better for every heterogeneous client.

81. The global model's final results

The notebook records approximately:

```
Home A federated → MAE 0.2245
Home B federated → MAE 0.7830
Home C federated → MAE 0.4611
```

compared with the corresponding local models:

```
Home A local → 0.1716
Home B local → 0.3815
Home C local → 0.4037
```

This custom FL procedure therefore performs poorly for some homes, particularly Home B.

That is **not evidence that Federated Learning itself doesn't work**.

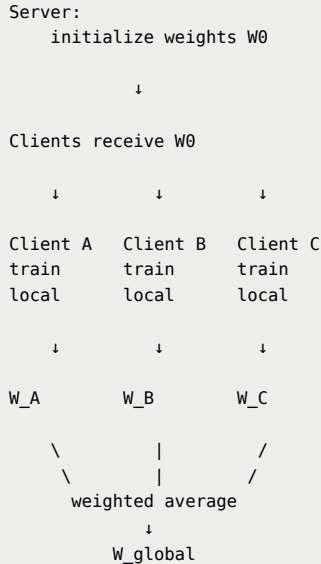
It means:

└ This particular surrogate FL algorithm is not handling the simulated Non-IID data particularly well.

That distinction is critical in a thesis.

82. True production-style FL with neural networks

If you used a neural network instead, the process would be much closer to classical FedAvg:



Then:

```
Round 2:
W_global
↓
clients
↓
```

```
local training
  ↓
updates
  ↓
average
```

This is the classical FL mental model.

83. Why FL is often demonstrated with neural networks

Neural-network parameters naturally form arrays/tensors:

```
weights
biases
```

which can be averaged numerically.

Random Forest requires specialized strategies because its internal structure is not a simple fixed parameter tensor.

For your defense, this is a very good question to expect:

| "Why didn't you directly average Random Forest weights?"

Answer:

| Because Random Forest does not expose a common fixed parameter vector analogous to neural-network weights. Its learned state consists of tree structures, split rules, thresholds, leaf values, and tree-specific structures, so direct FedAvg is not straightforward.

84. Important FL concepts you should know beyond this notebook

Client

A participant holding local data.

In your project:

```
Home A
Home B
Home C
```

Server

The coordinator.

It:

- initializes a model
 - distributes updates
 - receives client updates
 - aggregates them
 - produces a global model
-

Global model

The collaboratively learned model.

Local model

The model trained using one client's data.

Communication round

One complete cycle:

```
server → clients  
clients train  
clients → server  
server aggregates
```

Client drift

Because client data distributions differ, local training can move client models in different directions.

This is particularly important for Non-IID data.

Statistical heterogeneity

Differences in:

- feature distributions
- label distributions
- sample sizes
- behaviors
- usage patterns

between clients.

Your three homes are a simple simulation of this.

85. Does Federated Learning automatically guarantee privacy?

No.

This is another important FYP-defense point.

Federated Learning reduces the need to centralize raw data, but model updates themselves may potentially leak information.

Real systems may additionally use techniques such as:

```
Secure Aggregation
Differential Privacy
Encryption
Access Control
Trusted execution mechanisms
```

Those are beyond what this notebook implements.

So don't say:

| "Federated Learning mathematically guarantees privacy."

A better statement is:

| "Federated Learning reduces direct raw-data sharing by keeping training data on clients, but additional privacy mechanisms can be required for stronger protection."

86. Cell 6 — SHAP explainability

Now we move from:

```
What did the model predict?
```

to:

```
Why did the model predict it?
```

The guide defines SHAP as a way of quantifying how much each feature contributes to an individual prediction.

87. SHAP stands for

| SHapley Additive exPlanations

The theory comes from **Shapley values** in cooperative game theory.

Think of:

```
Prediction = team result
```

and each feature is:

```
player
```

SHAP tries to distribute the prediction's "credit" among the features.

88. SHAP equation

Conceptually:

$$f(x) = E[f(X)] + \sum_i \phi_i$$

where:

- $f(x)$ = model prediction
- $E[f(X)]$ = base/expected prediction
- ϕ_i = SHAP value for feature i

So:

$$\text{prediction} - \text{base} = \sum_i \text{SHAP}_i$$

The guide gives exactly this interpretation.

89. Example from your notebook

Home A:

```
Base value = 1.5741
Prediction = 0.7241
```

Difference:

$$0.7241 - 1.5741 = -0.8500$$

The SHAP values approximately sum to:

$$-0.8500$$

Therefore:

```
Base prediction
+
all feature contributions
=
final prediction
```

90. Positive vs negative SHAP

Suppose:

```
SHAP = +0.45
```

That feature pushed the prediction upward relative to the base value.

If:

```
SHAP = -1.14
```

it pushed the prediction downward.

It does **not** necessarily mean:

| "Increasing this feature in the real world will always increase consumption."

That would be a causal statement.

SHAP explains the model's behavior, not necessarily real-world causation.

This distinction is extremely important.

91. TreeExplainer

The notebook uses:

```
explainer = shap.TreeExplainer(model)
```

because the model is a tree ensemble.

TreeExplainer is designed specifically for tree-based models.

That makes it much more appropriate than treating the Random Forest like a generic black box.

92. Why only 500 samples?

The notebook uses:

```
sample_size = 500
```

instead of explaining the entire dataset.

Why?

SHAP can be computationally expensive.

So:

```
5665 rows
```

becomes:

```
500 sampled rows for analysis
```

This speeds things up.

But it means your "global SHAP importance" is actually:

| Importance estimated from the first 500 samples.

It is not the exact importance over the entire dataset.

93. Mean absolute SHAP

The notebook calculates:

```
mean_shap =  
    np.abs(shap_values).mean(axis=0)
```

That means:

1. take absolute value of each SHAP value
2. average across samples

Mathematically:

$\text{Importance}_j = \frac{1}{n} \sum_{i=1}^n |\phi_{ij}|$ $\text{Importance}_j = \frac{1}{n} \sum_{i=1}^n |\phi_{ij}|$

This tells us:

| How strongly does feature j tend to affect predictions, regardless of direction?

94. SHAP vs Random Forest feature importance

These are not the same thing.

Random Forest:

```
model.feature_importances_
```

is based on tree splitting criteria.

SHAP:

```
mean(|SHAP|)
```

is based on feature contribution to predictions.

So:

```
Random Forest importance
```

and:

```
SHAP importance
```

answer related but different questions.

95. Your SHAP results

For Home A:

```
rolling_mean_4 ≈ 0.6963
hour           ≈ 0.1594
lag_96        ≈ 0.1378
lag_1         ≈ 0.0752
```

For Home B:

```
rolling_mean_4 ≈ 1.4212
hour           ≈ 0.3145
lag_1         ≈ 0.2419
lag_4         ≈ 0.1122
```

For Home C:

```
rolling_mean_4 ≈ 0.8413
lag_1         ≈ 0.1401
hour          ≈ 0.0574
```

Across all three, `rolling_mean_4` dominates.

Again, because of the target leakage problem, you should **not interpret that as strong evidence that recent rolling average is inherently the most important real-world causal factor**.

It is partly a consequence of how the feature is constructed.

96. SHAP example in Home A

The notebook gives:

```
Base value      1.5741
```

Prediction	0.7241
------------	--------

Then:

rolling_mean_4	-1.1430
lag_96	+0.4533
hour	-0.2691
lag_1	+0.1023
...	

Interpretation:

```
Start at 1.5741
  ↓
rolling_mean_4 pushes down strongly
  ↓
lag_96 pushes up
  ↓
hour pushes down
  ↓
lag_1 pushes up slightly
  ↓
final = 0.7241
```

This is exactly the sort of explanation an operator can understand.

97. Important SHAP limitation: correlated features

Your features are highly related.

For example:

```
lag_1
rolling_mean_4
lag_4
rolling_std_4
```

all come from recent consumption.

Therefore several features may share predictive information.

So SHAP may distribute contribution among correlated variables in ways that are not uniquely identifiable.

Thus:

A SHAP value is an attribution under the model and feature-distribution assumptions, not a proof of causality.

98. The warning in the notebook

You got:

```
X has feature names, but RandomForestRegressor was fitted
without feature names
```

This happens because the model was originally trained on:

```
df[FEATURE_COLUMNS].values
```

which converts the DataFrame to a NumPy array.

Then SHAP later receives:

```
pd.DataFrame(...)
```

with feature names.

The model itself was fitted without names.

So Scikit-learn warns about the mismatch.

It is not necessarily changing the prediction, but it is a reproducibility/cleanliness issue.

99. Cell 7 — The Energy Agent

Now the project adds a decision-making layer.

This is:

```
class EnergyAgent:
```

The architecture is:

```
Observe
  ↓
Reason
  ↓
Act
```

The guide explicitly describes this observe → reason → act structure.

100. Is this actually an AI agent?

Technically, this notebook contains a:

| **rule-based agent / decision engine**

It is not an LLM agent.

There is:

```
no GPT
no LLM
no tool-calling model
no autonomous planning model
```

Instead, it uses deterministic Python rules.

The guide itself explains that this rule-based agent is intentionally simple and that a more sophisticated system could later replace it.

101. Agent initialization

You create:

```
agent_a = EnergyAgent(
    'Home A',
    'models/model_a.pkl'
)
```

The agent loads the Random Forest:

```
self.forecast_model = pickle.load(f)
```

Then it creates:

```
self.explainer =
    shap.TreeExplainer(self.forecast_model)
```

Therefore the agent has two main capabilities:

```
forecast
explain
```

102. Thresholds

The agent defines:

```
ANOMALY_THRESHOLD_LOW = 0.5
ANOMALY_THRESHOLD_HIGH = 0.75

FORECAST_HIGH_THRESHOLD = 3.5
FORECAST_VERY_HIGH = 5.0
```

So it has two types of evidence:

Anomaly evidence

```
0.5 → moderate  
0.75 → high
```

Forecast evidence

```
3.5 → high  
5.0 → very high
```

These are manually selected rules.

They are not learned automatically from data.

103. `observe()`

The observe stage does:

```
features_df = pd.DataFrame([features_dict])
```

Then:

```
prediction =  
    self.forecast_model.predict(X)[0]
```

So it obtains:

```
predicted consumption
```

Then it calculates SHAP:

```
shap_values =  
    self.explainer.shap_values(X)[0]
```

and sorts features by:

```
abs(shap_value)
```

So observation contains:

```
home_id  
predicted_kwh  
top_features  
raw_features
```

104. What "observe" means conceptually

The agent is observing the environment through:

```
current features
forecast
SHAP explanation
anomaly score
```

It is not physically reading a smart meter directly.

A real system would likely observe:

```
smart meter
IoT sensor
home gateway
weather
tariff
battery
solar generation
appliance states
```

Your educational version uses already-prepared feature data.

105. `reason()`

This is the actual decision logic.

First:

```
if anomaly_score > 0.75:
    HIGH_ANOMALY
```

If:

```
0.5 < anomaly <= 0.75
```

then:

```
MODERATE_ANOMALY
```

Then it checks predicted consumption:

```
if pred > 5.0:
    VERY_HIGH_FORECAST
elif pred > 3.5:
    HIGH_FORECAST
```

106. The SHAP-based rule

The agent also checks:

```
if feat == 'lag_1' and shap_val > 0.5:
```

and calls that:

```
RECENT_SURGE
```

This means the recent consumption value is strongly pushing the prediction upward.

This is a nice example of combining:

```
prediction
+
explanation
+
anomaly score
```

rather than relying only on one model output.

107. Severity logic

The priority order is:

```
HIGH_ANOMALY or VERY_HIGH_FORECAST
↓
CRITICAL

MODERATE_ANOMALY or HIGH_FORECAST
↓
WARNING

RECENT_SURGE
↓
ADVISORY

otherwise
↓
NORMAL
```

This is a rule hierarchy.

108. Why severity ordering matters

Suppose:

```
anomaly = 0.85
```

```
prediction = 6.2
```

Both are serious.

The rule gives:

```
CRITICAL
```

Suppose:

```
anomaly = 0.60  
prediction = 2.4
```

Then:

```
WARNING
```

Suppose:

```
anomaly = 0.20  
prediction = 2.0  
lag_1 SHAP = 0.8
```

Then:

```
ADVISORY
```

The agent therefore performs a kind of deterministic risk classification.

109. `act()`

After deciding severity, the agent produces an action.

Examples:

```
CRITICAL  
→ IMMEDIATE INSPECTION REQUIRED
```

or:

```
WARNING  
→ MONITOR CLOSELY
```

or:

```
NORMAL
→ NORMAL – NO ACTION REQUIRED
```

This is the final "action" component of the agent architecture.

110. The full agent cycle

The method:

```
run()
```

does:

```
observation = self.observe(...)
analysis = self.reason(...)
decision = self.act(...)
```

So:

Observe → Reason → ActObserve \rightarrow Reason \rightarrow Act

This is the central agent architecture.

111. The demonstration

The notebook tests two scenarios.

Normal case

```
anomaly_score = 0.2
```

Output:

```
NORMAL
NORMAL – NO ACTION REQUIRED
```

Predicted consumption:

```
0.96 kWh
```

Abnormal case

It manually changes:

```
anomaly_sample['lag_1'] = 9.5
anomaly_sample['rolling_mean_4'] = 7.2
```

and supplies:

```
anomaly_score=0.85
```

The result is:

```
CRITICAL
IMMEDIATE INSPECTION REQUIRED
```

with a predicted value around:

```
7.11 kWh
```

That is a very good demonstration of how the pieces connect.

112. But the agent has some limitations

The parameter:

```
anomaly_model_path=None
```

is never actually used to load an anomaly model.

So the class does **not** independently perform anomaly detection.

It expects:

```
anomaly_score
```

to be provided externally.

Conceptually:

```
Anomaly_detector
  ↓
anomaly_score
  ↓
EnergyAgent
```

rather than:

```
EnergyAgent
  ↓
```

```
internally runs anomaly model
```

113. Another limitation: arbitrary thresholds

These:

```
0.5
0.75
3.5
5.0
```

are manually chosen.

There is no statistical calibration shown.

For production use, you would ideally derive thresholds from:

- historical distributions
- operational requirements
- validation data
- cost of false positives
- cost of missed anomalies

114. Another limitation: "meter tampering"

The critical message says:

```
Check for faulty appliances, meter tampering, or unusual activity.
```

The model itself cannot establish meter tampering.

Anomaly detection means:

| "This pattern is unusual."

It does **not** prove:

| "The meter has been tampered with."

That is another important distinction for an FYP presentation.

115. How all the machine-learning pieces connect

Now let's put everything together.

Forecasting model

Answers:

| What is likely to happen next?

Random Forest

Anomaly detector

Answers:

| Does the current pattern look unusual?

Isolation Forest

SHAP

Answers:

| Why did the forecasting model produce that prediction?

TreeExplainer

Agent

Answers:

| Given the prediction, anomaly state, and explanation, what should we recommend?

Rule-based decision logic

Federated Learning

Answers:

| How can multiple homes collaborate without centralizing their raw training data?

local training + aggregation

116. The conceptual architecture of your FYP

Your project can therefore be understood as five layers.

Layer 1 — Data

Home A
Home B
Home C

Non-IID consumption data.

Layer 2 — Intelligence

Random Forest
Isolation Forest

Forecast + anomaly detection.

Layer 3 — Privacy

Federated Learning

Collaborative learning architecture.

Layer 4 — Explainability

SHAP

Understand the prediction.

Layer 5 — Decision

Energy Agent

Convert model outputs into recommendations.

The guide's later sections then wrap these capabilities behind an API and dashboard.

117. What the companion API/dashboard sections are doing

Although they are not executable cells in your uploaded notebook, the companion guide extends the design into:

```
FastAPI
  ↓
REST endpoints
  ↓
Streamlit dashboard
```

The guide defines endpoints such as:

```
/forecast/{home_id}
/anomalies/{home_id}
/explanation/{home_id}
/recommendation/{home_id}
/comparison
/federated/status
```

and then presents the outputs in Streamlit.

So the broader architecture becomes:

```
ML / FL backend
  |
FastAPI REST API
  |
Streamlit UI
  |
User/operator
```

118. FastAPI concept

FastAPI turns Python functions into HTTP endpoints.

For example:

```
GET /forecast/home-a
```

might return:

```
{
  "predicted_consumption_kwh": 3.82
}
```

The point is that the model no longer exists only as a notebook script.

It becomes a reusable service.

The guide explains this exact transition from Python code to API service.

119. Streamlit concept

Streamlit provides a Python-based web interface.

Instead of building:

```
HTML
CSS
JavaScript
React
backend
```

you can create an interactive dashboard mostly in Python.

The guide's dashboard contains sections for:

```
Overview
Consumption
Forecasting
Anomaly Detection
SHAP
Federated Learning
Agent recommendation
```

120. What happens during a real user request

Imagine the user selects Home A.

The complete flow is conceptually:

```
User selects Home A
↓
dashboard/API
↓
load latest feature vector
↓
Random Forest
↓
prediction
↓
Isolation Forest
↓
anomaly state
↓
SHAP
↓
important factors
↓
Agent
↓
recommendation
↓
dashboard
```

So the user might see:

```
Predicted consumption: 4.21 kWh  
Anomaly score: 0.83  
Severity: CRITICAL
```

```
Why?  
rolling_mean_4  
lag_1  
hour
```

```
Recommendation:  
Inspect high-load appliances.
```

That is the complete smart-energy intelligence concept.

121. The biggest technical issues you should know

For your FYP, I would memorize these because they are exactly the sort of things an examiner could challenge.

Issue 1 — Forecast target mismatch

The comments say:

```
predict next consumption
```

but the target is:

```
consumption_kwh
```

not:

```
consumption_kwh.shift(-1)
```

Issue 2 — Target leakage

The current consumption is included in:

```
rolling_mean_4  
rolling_mean_96  
rolling_std_4
```

while `consumption_kwh` is also the target.

This can artificially inflate performance.

Issue 3 — Notebook's FL is not classical FedAvg

The code does not average model parameters.

It averages local predictions on a shared feature set and trains a new model on them.
That is better described as a **prediction-aggregation/distillation-style simulation**.

Issue 4 — Privacy claim is not fully satisfied

The code builds:

```
X_shared
```

from actual client training data.

Therefore private feature rows are being shared centrally in the simulation.

That conflicts with a strict interpretation of:

| raw data never leaves the home.

Issue 5 — The "initial global model" is not actually initialized

`global_params` is `None` during Round 1.

Issue 6 — `shared_y` is unused

It is constructed but does not participate in aggregation.

Issue 7 — Local target mixing is custom

```
0.7 * local_y + 0.3 * global_prediction
```

is a custom heuristic, not standard FedAvg.

Issue 8 — Anomaly evaluation uses the same dataset used for fitting

This is not ideal for rigorous generalization testing.

Issue 9 — `contamination=0.02` drives the number of alerts

You injected 20 anomalies, but the algorithm is configured to find roughly 113.

That explains the high false-positive count.

Issue 10 — SHAP importance is affected by feature leakage

The fact that `rolling_mean_4` dominates SHAP is partly consistent with it containing the target itself.

Issue 11 — The agent is rule-based

It is not an LLM-based autonomous agent.

Issue 12 — The agent thresholds are hand-written

They are not learned/calibrated.

122. What a corrected forecasting formulation should look like

A much cleaner setup would be:

```
At time t:

lag_1      = y[t]
lag_4      = y[t-3]
lag_96     = y[t-95]
rolling_mean = based only on observations <= t
hour       = hour(t)
day_of_week = day(t)

Target:

y[t+1]
```

Then:

$X_t \rightarrow y_{t+1} | X_t$

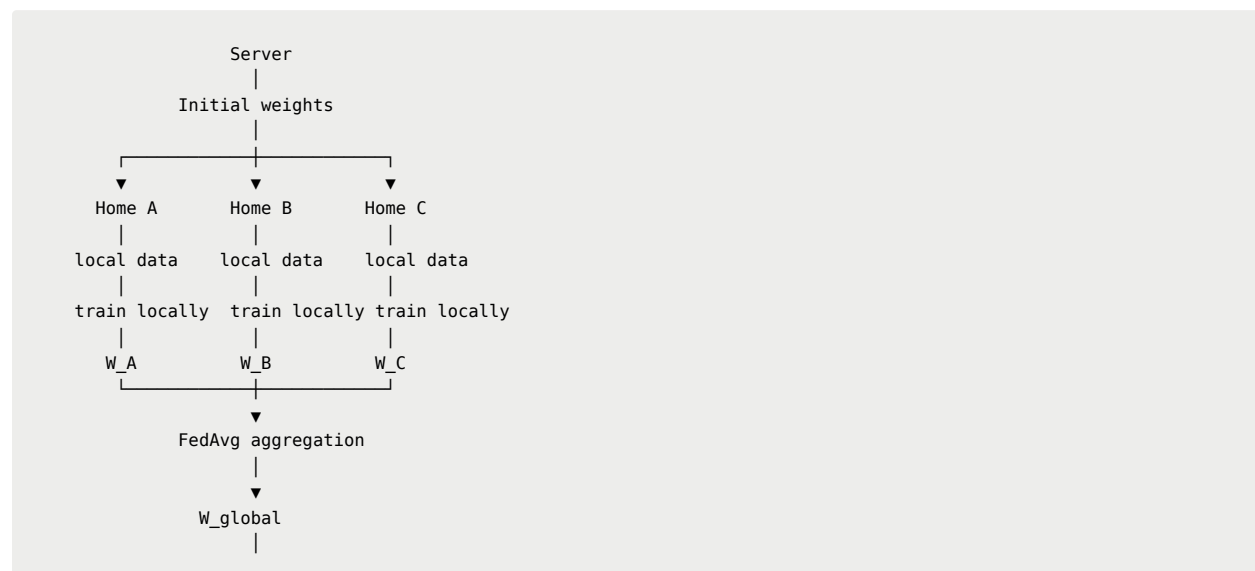
rather than:

$X_t \rightarrow y_t | X_t$

with target information embedded in the features.

123. What a more authentic FL architecture should look like

For example, with a neural network:



▼
next FL round

Crucially:

```
raw data:  
Home A → stays at A  
Home B → stays at B  
Home C → stays at C
```

Only model updates are communicated.

124. What your current implementation is actually demonstrating

The honest description would be:

"A pedagogical simulation of federated learning for Random Forest forecasting using local client models, prediction aggregation, and a server-side surrogate global model."

That is much more technically accurate than simply calling it:

"FedAvg Random Forest."

125. One complete example from start to finish

Suppose Home A currently has:

```
lag_1 = 4.8  
lag_4 = 4.2  
lag_96 = 3.9  
rolling_mean_4 = 4.6  
hour = 19  
...
```

Step 1 — Forecast

Random Forest produces:

5.4 kWh

Step 2 — Anomaly detection

Isolation Forest gives a high anomaly score:

0.84

Step 3 — SHAP

Suppose:

```
lag_1          +0.7
rolling_mean_4 +1.0
hour           +0.4
```

This says the recent usage is strongly pushing the forecast upward.

Step 4 — Agent reasoning

Rules detect:

```
HIGH_ANOMALY
VERY_HIGH_FORECAST
RECENT_SURGE
```

Step 5 — Severity

```
CRITICAL
```

Step 6 — Action

```
IMMEDIATE INSPECTION REQUIRED
```

The whole project has now produced:

```
Prediction
+
Anomaly
+
Explanation
+
Recommendation
```

That is the real value of combining these components.

126. The difference between the five ML/AI concepts

This is probably the single most useful table to memorize for your FYP defense.

Concept	Main question	Technique in project
Forecasting	What will consumption be?	Random Forest Regressor
Anomaly Detection	Is this behavior unusual?	Isolation Forest

Concept	Main question	Technique in project
Federated Learning	How can clients learn collaboratively without centralizing raw data?	custom FL simulation
Explainable AI	Why did the model make this prediction?	SHAP
Agent	What should we do about the result?	Rule-based decision engine

127. Difference between supervised, unsupervised, and federated learning

These are often confused.

Supervised learning

You have:

```
x → y
```

Example:

```
features → actual consumption
```

Your forecasting model is supervised.

Unsupervised learning

You don't necessarily have target labels during model fitting.

Example:

```
normal patterns → discover unusual observations
```

Your Isolation Forest is unsupervised.

Federated Learning

Federated Learning is not simply another supervised/unsupervised algorithm.

It describes **where and how training happens**.

You can perform:

```
supervised FL
unsupervised FL
self-supervised FL
```

So:

```
Supervised/unsupervised
  ↓
learning objective

Federated
  ↓
training architecture
```

This distinction is extremely important.

128. Difference between forecasting and anomaly detection

Forecasting:

```
Input:
historical context

Output:
future numerical value
```

Example:

```
Next consumption = 3.8 kWh
```

Anomaly detection:

```
Input:
current pattern

Output:
normal/anomalous
```

Example:

```
Current pattern = anomalous
```

So they solve different problems.

129. Difference between Random Forest and Isolation Forest

Despite both containing the word "Forest", they are fundamentally different.

Random Forest

Purpose:

prediction

Your model:

RandomForestRegressor

It builds trees to predict a continuous target.

Isolation Forest

Purpose:

anomaly detection

Your model:

IsolationForest

It isolates unusual observations through random partitioning.

130. Why SHAP is not another prediction model

SHAP does not replace Random Forest.

Instead:

Random Forest
↓
prediction

and then:

SHAP
↓
explanation of prediction

So SHAP is an **interpretability layer**.

131. Why the agent is not another ML model

The agent takes existing outputs:

```
forecast
anomaly score
SHAP
```

and applies decision logic.

Therefore:

```
ML models → generate evidence
Agent      → interprets evidence and recommends action
```

This is a very useful architectural distinction.

132. How the whole project maps to an FYP story

A good way to present the project verbally is:

"Each smart home is treated as a federated client with private energy-consumption data. The system extracts temporal features such as lagged consumption and rolling statistics. A local Random Forest performs consumption forecasting, while Isolation Forest detects unusual usage patterns. Federated learning is then used to simulate collaboration among heterogeneous homes. SHAP provides feature-level explanations for predictions, and a rule-based energy agent combines forecasts and anomaly information into an actionable recommendation."

That is the coherent story.

133. What your examiner may ask

You should be prepared to answer:

Why is the data Non-IID?

Because each home's consumption distribution differs through its base load and noise characteristics.

Why use lag features?

Because historical energy usage contains temporal information useful for predicting future consumption.

Why lag₉₆ ?

Because 96 fifteen-minute intervals equal 24 hours.

Why use time-based splitting?

Because random splitting can introduce future information into training.

What is MAE?

Average absolute prediction error in kWh.

What is RMSE?

Square-root of mean squared error, which penalizes large errors more heavily.

What is overfitting?

Learning training-specific noise rather than generalizable patterns.

What is FedAvg?

Aggregating local model parameters, usually using a data-size-weighted average.

Is your implementation true FedAvg?

Not in the standard parameter-averaging sense; it is a pedagogical prediction-aggregation simulation for Random Forest.

Why can't Random Forest weights be simply averaged?

Because tree structures and learned split rules are not aligned parameter tensors like neural networks.

Does FL guarantee privacy?

No. It reduces direct raw-data sharing, but stronger privacy may require additional mechanisms.

What does a SHAP value mean?

How much a feature contributes to moving a particular model prediction away from its baseline prediction.

Does SHAP prove causation?

No. It explains model behavior, not causal effects.

Is your agent an LLM agent?

No. It is a deterministic rule-based agent demonstrating the observe-reason-act architecture.

134. The most important corrections I would make before calling this a rigorous FYP implementation

Conceptually, the upgraded system should change four things first.

Forecasting

Change from:

```
features_t → consumption_t
```

to:

```
features_t → consumption_(t+1)
```

and ensure no future/target information enters the features.

Federated Learning

Use a model whose parameters can genuinely be aggregated, such as:

```
neural network
```

and implement actual FedAvg:

$$w_{t+1} = \sum_k \frac{n_k}{N} w_k^{t+1} = \sum_k \frac{n_k}{N} w_k^{t+1}$$

Privacy

Do not create:

```
X_shared
```

from private client records.

If a public aggregation/evaluation set is needed, it should genuinely be public or otherwise independently sourced.

Evaluation

Use:

```
time-based validation
held-out test data
multiple FL rounds
client-level metrics
global metrics
communication cost
privacy assumptions
```

and ideally compare:

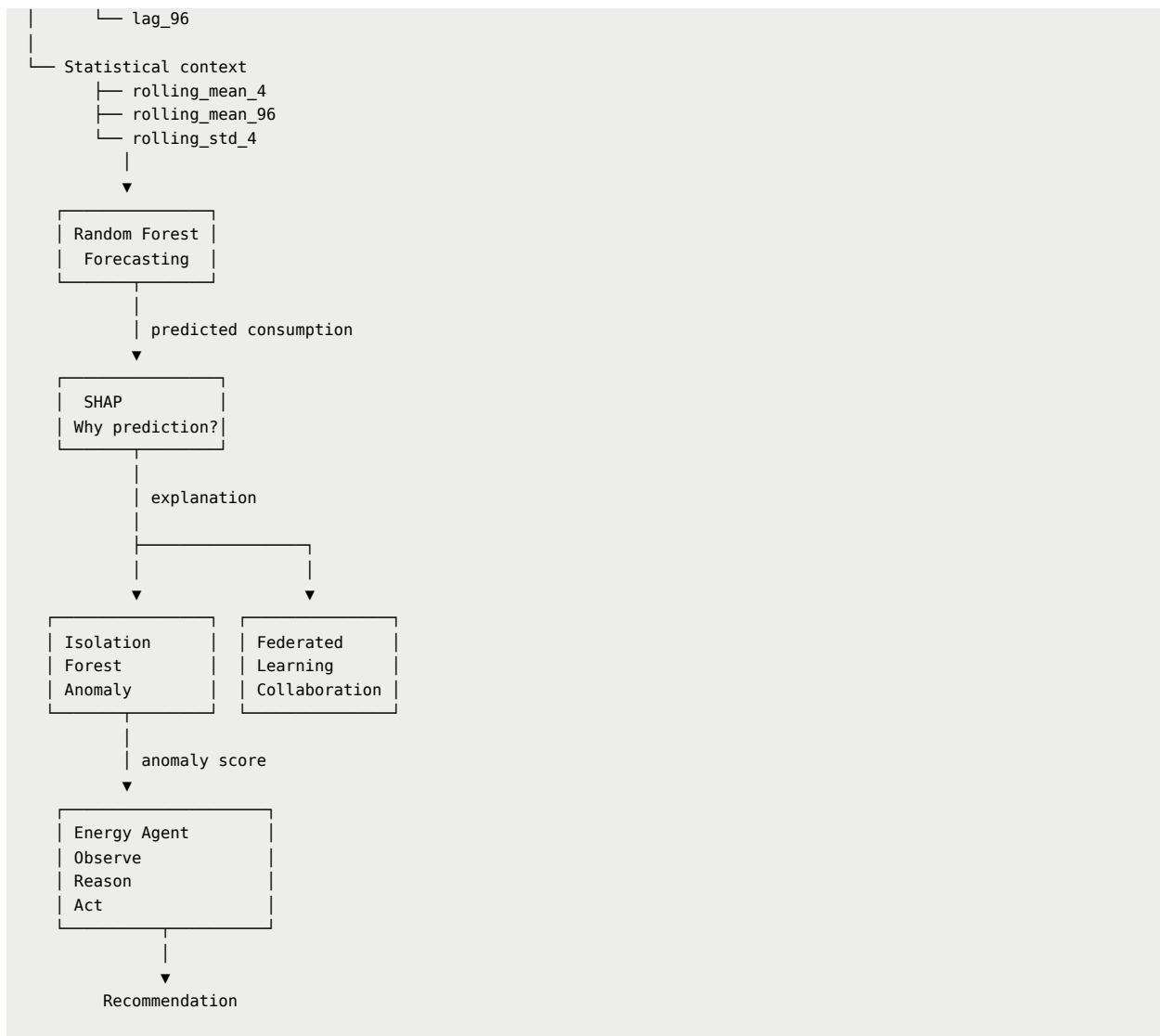
```
Local
Centralized
Federated
```

under the same experimental protocol.

135. Final mental model

You should think about the project this way:

```
DATA
|
|— Time information
|   |— hour
|   |— day_of_week
|   |— weekend
|   |— month
|
|— Historical information
|   |— lag_1
|   |— lag_4
```



The companion guide's run sequence follows this same progression from data generation through forecasting, anomaly detection, local/centralized comparison, FL, SHAP, agent, API, and dashboard.

The single biggest thing to understand

The project is **not just "an ML model."**

It is a layered system:

Data → Forecasting → Anomaly Detection → Federated Collaboration → Explainability → Decision Agent

And the most important technical caveat is:

The notebook demonstrates the FL idea, but its current Random-Forest implementation is not standard FedAvg and is not fully privacy-preserving as coded.

\text{The notebook demonstrates the FL idea, but its current Random-Forest implementation is not standard FedAvg and is not fully privacy-preserving as coded.} }

That distinction will make your understanding much stronger than simply memorizing what each code block does.